Proyecto TLP02-2026

Gramáticas formales y lenguaje natural: experimentando los límites de la jerarquía de Chomsky

Introducción

Este proyecto busca que los estudiantes vivan de primera mano la frontera entre los lenguajes formales y el lenguaje natural, y que comprendan por qué las gramáticas libres de contexto (CFG) son suficientes para modelar lenguajes de programación pero se quedan cortas al intentar modelar el español. En lugar de limitarnos a observar que un parser falla con frases en lenguaje natural, el objetivo es que ustedes identifiquen con precisión en qué punto la CFG se vuelve insuficiente, por qué razones formales ocurre esto, y cómo se compara una gramática formal frente a las técnicas modernas de Procesamiento de Lenguaje Natural (NLP) basadas en estadística y aprendizaje profundo.

El parser o analizador sintáctico se encarga de revisar el texto de entrada con base en una sintaxis definida, utilizando métodos sistemáticos basados en gramáticas libres de contexto. Es la pieza del compilador que valida si una secuencia de tokens pertenece al lenguaje descrito por la gramática y, a partir de ella, construye el árbol de análisis sintáctico que servirá para fases posteriores como el análisis semántico y la generación de código.

La idea de este proyecto es mantener el corazón de compiladores y teoría de lenguajes de programación (gramáticas formales, parsers descendentes, árboles de derivación), pero abrir una ventana al mundo del NLP y la IA, poniendo a pueba hasta dónde llega una CFG y qué pueden hacer los modelos modernos que las CFG no pueden

El proyecto se desarrollará en dos fases complementarias, en grupos de 4 a 5 integrantes, combinando investigación teórica, implementación práctica y comparación crítica con herramientas actuales de NLP e IA. Los grupos deben estar conformados al inicio de la Fase 1 y mantenerse durante todo el proyecto.

Fase 1 – Lenguajes formales en la frontera del lenguaje natural (40%)

Objetivos:

- Implementar un parser formal (LL(1) o descendente recursivo) para un subconjunto de un lenguaje de programación estándar (C, C++, C#, Java, etc.) y comprobar su correcto funcionamiento.
- Intentar extender la gramática libre de contexto del parser para capturar fenómenos del español, tales como concordancia de género y número, cláusulas subordinadas y modificadores.
- Identificar y documentar, con precisión formal, en qué punto exacto y por qué razones la CFG se vuelve insuficiente para modelar esos fenómenos.
- Conectar la experiencia práctica con la jerarquía de Chomsky y las limitaciones formales vistas en clase.

En resumen esta fase consiste en: implementar un parser clásico para un lenguaje de programación, demostrar que funciona correctamente, y luego llevarlo al límite intentando extenderlo al español. El entregable incluye el código del parser, una demo donde se vea tanto el funcionamiento correcto sobre el lenguaje formal como los puntos de falla al intentar procesar oraciones en español, y un informe que explique formalmente y resumidamente por qué esas fallas no son accidentales sino consecuencia de las limitaciones de las CFG.

Actividades:

- Investigar, con fundamento teórico, las diferencias clave entre lenguajes formales y lenguajes naturales (estructura, ambigüedad, recursividad, dependencias de contexto, concordancia).
- Implementar en Python un parser LL(1) o descendente recursivo que analice un subconjunto definido de un lenguaje de programación estándar (puede ser un subconjunto de C, un lenguaje aritmético con estructuras de control, o algún otro acordado previamente con el docente). El parser debe producir un árbol de análisis sintáctico y manejar errores sintácticos básicos.
- Definir un conjunto de pruebas que demuestre que el parser funciona correctamente sobre entradas válidas y rechaza correctamente entradas inválidas del lenguaje formal.
- Intentar extender la CFG del parser para que acepte oraciones del español que presenten al menos los siguientes fenómenos: concordancia de género y número entre sustantivo-adjetivo y sujeto-verbo, al menos un tipo de cláusula subordinada, y modificadores (adjetivos, adverbios, frases preposicionales)
- Documentar sistemáticamente los puntos exactos donde la CFG falla: identificar la regla que falla, la clase de fenómeno lingüístico que la rompe (por ejemplo, dependencias cruzadas, ambigüedad estructural, concordancia a distancia), y relacionarlo con el nivel correspondiente de la jerarquía de Chomsky.
- Realizar una demo apoyada en una librería de NLP (NLTK o spaCy) que procese las mismas oraciones que su gramática extendida intentó analizar, para evidenciar la diferencia en capacidad de análisis.

Entregables:

- Informe técnico (5-7 páginas) que incluya: marco teórico sobre jerarquía de Chomsky y limitaciones de las CFG, descripción de la gramática del parser formal, descripción de los intentos de extensión al español, análisis formal de los puntos de falla, y comparación con la salida de NLTK/spaCy sobre las mismas oraciones.
- Código fuente documentado del parser formal, los intentos de extensión al español, y los casos de prueba utilizados. Debe entregarse en un repositorio Git público o privado (compartido con el docente) con un README que explique cómo ejecutar el código. O subir el código en E-campus

- Presentación grupal (15-20 minutos) explicando los hallazgos, con demostración en vivo tanto del parser funcionando sobre el lenguaje formal como de los puntos de falla al intentar procesar español.

Fecha de entrega: 2 de noviembre de 2026, a más tardar 23:59.

Fechas de presentaciones: martes 3 y jueves 5 de noviembre. El orden de los grupos se definirá por sorteo o inscripción el día lunes previo a las presentaciones.

Fase 2 – Parser para el español y comparación con NLP moderno (60%)

Objetivos:

- Diseñar e implementar un parser propio para un subconjunto controlado del español, capaz de generar árboles de análisis sintáctico visuales para cada oración procesada.
- Evaluar cuantitativamente el desempeño del parser propio frente a herramientas modernas de NLP (spaCy, NLTK o HuggingFace Transformers) usando un conjunto de prueba definido por el grupo.
- Explorar la capacidad de un modelo de lenguaje grande (LLM) para seguir especificaciones gramaticales formales cuando se le proporcionan explícitamente las reglas de producción.
- Producir hallazgos respaldados con evidencia experimental (datos, no observaciones cualitativas) sobre las diferencias entre parsers formales, modelos estadísticos y LLMs.

Actividades:

- Definir formalmente una gramática libre de contexto para un subconjunto controlado del español. Debe cubrir, como mínimo: oraciones Sujeto-Verbo-Objeto, modificadores (adjetivos, adverbios, frases preposicionales), al menos dos tiempos verbales, concordancia básica dentro de las limitaciones de las CFG, y un vocabulario reducido pero suficiente para construir un conjunto de prueba variado.
- Implementar un parser descendente recursivo (o LL(1), según el diseño del grupo) para esa gramática, que valide frases y genere árboles de análisis sintáctico visuales. La visualización puede hacerse con graphviz, nltk.Tree, matplotlib o cualquier librería equivalente.
- Construir un conjunto de prueba con tres niveles de complejidad lingüística: (1) oraciones simples que la gramática debe aceptar, (2) oraciones intermedias con modificadores y variaciones, y (3) oraciones complejas con fenómenos que probablemente excedan la gramática. El conjunto debe tener al menos 60 oraciones distribuidas entre los tres niveles, con etiquetado de la salida esperada.
- Procesar el mismo conjunto con al menos dos herramientas modernas de NLP (por ejemplo spaCy y HuggingFace Transformers), y comparar los resultados con los del parser propio.

- Calcular y reportar métricas cuantitativas: precisión, cobertura (recall), tasa de rechazo correcto, y una clasificación de errores (falsos positivos, falsos negativos, errores de etiquetado). Los resultados deben presentarse en tablas y gráficas.
- Utilizar un LLM vía API (por ejemplo la API de Anthropic con Claude, OpenAI, Gemini u otro servicio equivalente) proporcionándole explícitamente las reglas de producción de su gramática formal en el prompt, y pedirle que analice oraciones del conjunto siguiendo estrictamente esas reglas. Documentar: **(a)** si el LLM respeta las reglas formales o se desvía hacia su conocimiento general del español, **(b)** en qué casos acierta y en cuáles falla, **(c)** si la precisión mejora cuando el prompt incluye ejemplos (few-shot) frente a solo las reglas (zero-shot).
- Analizar las diferencias observadas en términos de robustez, manejo de ambigüedad, escalabilidad, costo computacional y aplicabilidad práctica de cada enfoque.

Entregables:

- Informe final (7-10 páginas) que documente: diseño de la gramática formal, implementación del parser, descripción del set de prueba, metodología de comparación, tablas y gráficas con los resultados cuantitativos, análisis de la comparación con spaCy/NLTK/HuggingFace, análisis del comportamiento del LLM frente a reglas formales, y conclusiones respaldadas por los datos.
- Código fuente documentado del parser, el script de visualización de árboles, el set de prueba, el script de comparación con las herramientas de NLP, y los prompts utilizados con el LLM (incluyendo las respuestas obtenidas en un archivo de logs). Todo en un repositorio Git con README que explique cómo reproducir los experimentos o entregado a través del e-campus.
- Presentación final (15-20 minutos) con demostración en vivo del parser procesando oraciones del set, mostrando los árboles sintácticos generados y presentando las tablas comparativas con datos concretos. Se valorará especialmente que las conclusiones estén respaldadas por evidencia experimental y no solo por observaciones cualitativas.

Fecha de entrega: 17 de noviembre de 2026, a más tardar 23:59.

Fechas de presentaciones: martes 17 y jueves 19 de noviembre. El orden de los grupos se definirá por sorteo o inscripción el día lunes previo a las presentaciones.

Criterios de evaluación

Cada fase se evaluará considerando los siguientes criterios:

- **Correctitud técnica (35%):** que el parser funcione según lo especificado, que la gramática esté bien definida, y que el código sea reproducible.
- **Profundidad del análisis (30%):** que lo presentado/expuesto conecte la experiencia práctica con los fundamentos teóricos vistos en clase (jerarquía de Chomsky, limitaciones de las CFG, propiedades de los lenguajes formales).

- **Rigor experimental (20%):** especialmente en la Fase 2, que las conclusiones estén respaldadas por datos cuantitativos, no por impresiones.
- **Calidad de la presentación y demo (15%):** claridad en la exposición, funcionamiento de la demo en vivo, capacidad para responder preguntas del docente y los compañeros.

Consideraciones finales

- El proyecto completo representa el 25% de la nota final de la asignatura, distribuido en Fase 1 (40% del proyecto) y Fase 2 (60% del proyecto).
- Los grupos deben estar conformados por 4 a 5 integrantes y mantenerse estables durante ambas fases. Cualquier cambio de integrantes debe ser autorizado por el docente.
- La asistencia a las presentaciones es obligatoria para el 50% + 1 del los integrantes del grupo.
- El uso de LLMs como asistentes de programación durante el desarrollo del proyecto está permitido, pero los estudiantes deben ser capaces de explicar y defender cada línea de código entregado durante la presentación. En la Fase 2, el uso del LLM vía API es parte explícita del experimento y debe documentarse.